电机的驱动与编码器读取-USART

1.1 开篇说明

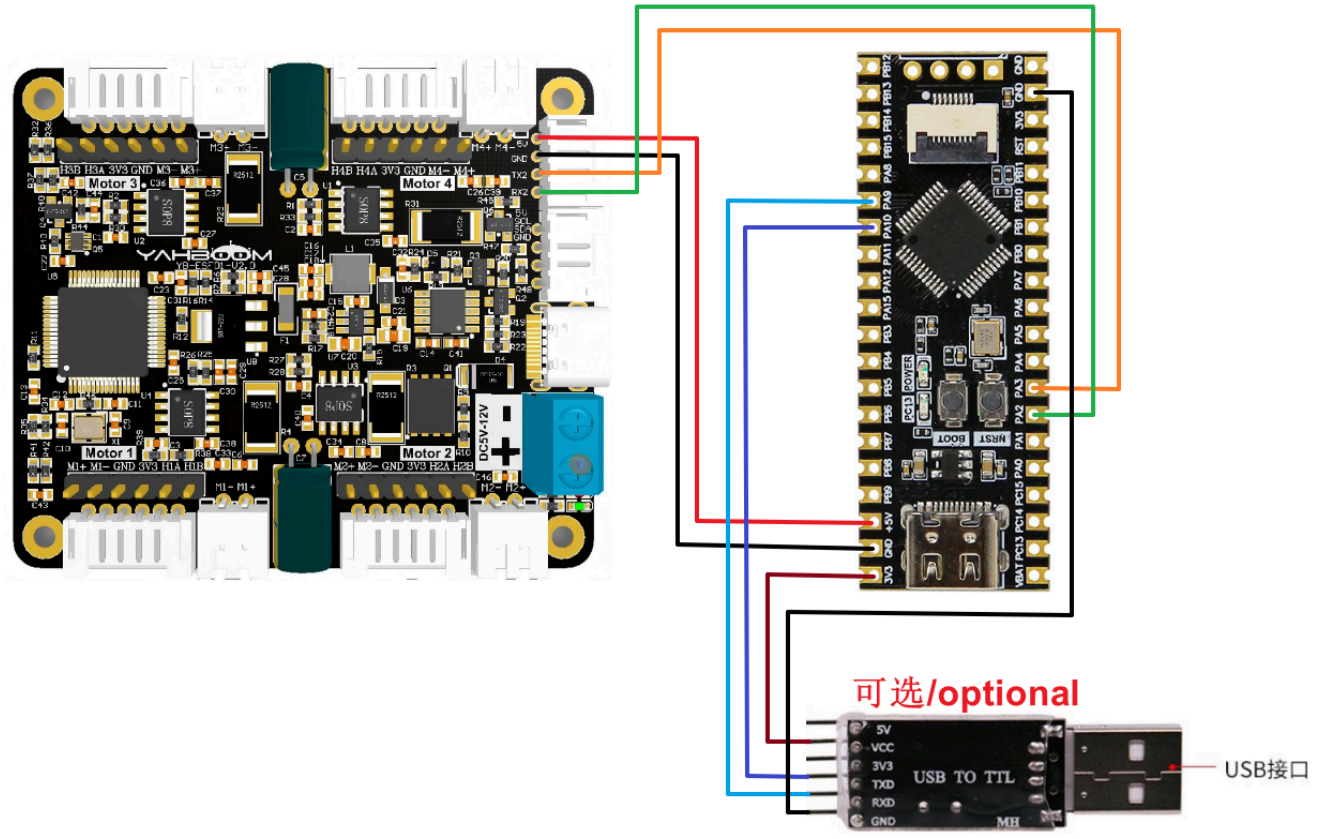
请先阅读《常用电机介绍以及用法》，了解清楚自己现使用的电机参数、接线方式、供电电压。以免造成烧坏主板或者电机的后果。

I2C与串口通讯不能共用，只能选择其中一个。STM32C8T6与RCT6都可兼容使用本工程。

4个电机接口对应小车的关系如下：

M1 -> 左上电机(小车的左前轮)
M2 -> 左下电机(小车的左后轮)
M3 -> 右上电机(小车的右前轮)
M4 -> 右下电机(小车的右后轮)

硬件接线：



四路电机驱动板	STM32C8T6/STM32RCT6
RX2	PA2
TX2	PA3
GND	GND
5V	5V

电机	四路电机驱动板(Motor)
M2	M-
V	3V3
A	H1B
B	H1A
G	GND
M1	M+

此处接USB转TTL串口模块，是用于打印处理过后的编码器数据的。如果使用的是在本店购买的STM32，可以直接通过TYPE-C接口与电脑USB口相连，也能使用串口打印，就不需要再接USB转TTL串口模块了。

USB TO TTL	STM32C8T6/STM32RCT6
VCC	3V3
GND	GND
RXD	PA9
TXD	PA10

串口配置：**波特率115200、无奇偶校验、无硬件流控、1位停止位**

注：此处的串口用于在串口助手上打印数据，不是用于与驱动板通讯

1.2 代码解释

```
#define UPLOAD_DATA 2  // 1:接收总的编码器数据 2:接收实时的编码器 1: Receive total encoder data 2: Receive real-time encoder data

#define MOTOR_TYPE 1   //1:520电机 2:310电机 3:测速码盘TT电机 4:TT直流减速电机 5:L型520电机
                       //1:520 motor 2:310 motor 3:speed code disc TT motor 4:TT DC reduction motor 5:L type 520 motor
```

- UPLOAD_DATA：用于设置接收电机编码器的数据，设置1为接收编码器总脉冲数，2为10ms的实时脉冲数据。
- MOTOR_TYPE：用于设置使用的电机类型，根据自己现使用的电机对照着注释修改对应的数字即可，不需要再修改代码其余部分。

如果是需要驱动电机并观察数据，只修改程序开头的这两个数字就可以了，代码的其余部分不需要做任何更改。

```
    #if MOTOR_TYPE == 1
    send_motor_type(1);//配置电机类型 Configure motor type
    delay_ms(100);
    send_pulse_phase(30);//配置减速比 查电机手册得出    Configure the reduction ratio. Check the motor manual to find out
```

```
    delay_ms(100);
    send_pulse_line(11);//配置磁环线 查电机手册得出 Configure the magnetic ring wire. Check the motor manual to get the result.
    delay_ms(100);
    send_wheel_diameter(67.00);//配置轮子直径,测量得出        Configure the wheel diameter and measure it
    delay_ms(100);
    send_motor_deadzone(1900);//配置电机死区,实验得出 Configure the motor dead zone, and the experiment shows
    delay_ms(100);

    #elif MOTOR_TYPE == 2
    send_motor_type(2);
    delay_ms(100);
    send_pulse_phase(20);
    delay_ms(100);
    send_pulse_line(13);
    delay_ms(100);
    send_wheel_diameter(48.00);
    delay_ms(100);
    send_motor_deadzone(1600);
    delay_ms(100);

    #elif MOTOR_TYPE == 3
    send_motor_type(3);
    delay_ms(100);
    send_pulse_phase(45);
    delay_ms(100);
    send_pulse_line(13);
    delay_ms(100);
    send_wheel_diameter(68.00);
    delay_ms(100);
    send_motor_deadzone(1250);
    delay_ms(100);

    #elif MOTOR_TYPE == 4
    send_motor_type(4);
    delay_ms(100);
    send_pulse_phase(48);
    delay_ms(100);
    send_motor_deadzone(1000);
    delay_ms(100);

    #elif MOTOR_TYPE == 5
    send_motor_type(1);
    delay_ms(100);
    send_pulse_phase(40);
    delay_ms(100);
    send_pulse_line(11);
    delay_ms(100);
    send_wheel_diameter(67.00);
    delay_ms(100);
    send_motor_deadzone(1900);
```

```
    delay_ms(100);
    #endif
```

这里用于存储本店在售电机的参数，通过修改上方的MOTOR_TYPE参数，就可以实现一键配置。正常情况下使用本店的电机不要修改此处的代码。如果使用的是自己的电机，或者说某项数据需要根据自己的需求来修改，那么可以查看文档《4路电机驱动板控制指令》来了解每一项指令的具体含义。

```
    while(1)
    {
        if(times>=250)
        {
            #if MOTOR_TYPE == 4
            Car_Move_PWM();
            #else
            Car_Move();
            #endif
            times = 0;
        }
        if(g_recv_flag == 1)
        {
            g_recv_flag = 0;

            #if UPLOAD_DATA == 1
                Deal_data_real();

printf("M1:%d,M2:%d,M3:%d,M4:%d\r\n",Encoder_Now[0],Encoder_Now[1],Encoder_Now[2],Encoder_No
w[3]);
            #elif UPLOAD_DATA == 2
                Deal_data_real();

printf("M1:%d,M2:%d,M3:%d,M4:%d\r\n",Encoder_Offset[0],Encoder_Offset[1],Encoder_Offset[2],E
ncoder_Offset[3]);
            #elif UPLOAD_DATA == 3
                Deal_data_real();

printf("M1:%.2f,M2:%.2f,M3:%.2f,M4:%.2f\r\n",g_Speed[0],g_Speed[1],g_Speed[2],g_Speed[3]);
            #endif
        }
    }
```

程序中设置了一个10ms的定时器，每一次的定时器中断触发，都会使times这个变量加一，当达到250次，即2.5秒的时候，改变小车的运动状态。如果电机类型选择的是4，即无编码器的电机，那使用pwm版本的小车状态切换函数。同时读取驱动板发送上来的数据，并一边将数据打印出来。

```
//检验从驱动板发送过来的数据，符合通讯协议的数据则保存下来
//Check the data sent from the driver board, and save the data that meets the communication
protocol
void Deal_Control_Rxtemp(uint8_t rxtemp)
{
    static u16 step = 0;
    static u8 start_flag = 0;
```

```
    if(rxtemp == '$' &&     start_flag == 0)
    {
        start_flag = 1;
        memset(g_recv_buff,0,RXBUFF_LEN);//清空数据 Clear data
    }

    else if(start_flag == 1)
    {
            if(rxtemp == '#')
            {
                start_flag = 0;
                step = 0;
                g_recv_flag = 1;
                memcpy(g_recv_buff_deal,g_recv_buff,RXBUFF_LEN); //只有正确才会赋值 Only
correct ones will be assigned
            }
            else
            {
                if(step > RXBUFF_LEN)
                {
                    start_flag = 0;
                    step = 0;
                    memset(g_recv_buff,0,RXBUFF_LEN);//清空接收数据   Clear received data
                }
                else
                {
                    g_recv_buff[step] = rxtemp;
                    step++;
                }
            }
    }

}

//将从驱动板保存到的数据进行格式处理，然后准备打印
//Format the data saved from the driver board and prepare it for printing
void Deal_data_real(void)
{
    static uint8_t data[RXBUFF_LEN];
    uint8_t  length = 0;

    //总体的编码器    Overall encoder
     if ((strncmp("MAll",(char*)g_recv_buff_deal,4)==0))
    {
        length = strlen((char*)g_recv_buff_deal)-5;
        for (uint8_t i = 0; i < length; i++)
        {
            data[i] = g_recv_buff_deal[i+5]; //去掉冒号 Remove the colon
        }
                data[length] = '\0';
```

```
                char* strArray[10];//指针数组 长度根据分割号定义  char 1字节   char* 4字节
Pointer array The length is defined by the split number char 1 byte char* 4 bytes
                char mystr_temp[4][10] = {'\0'};
                splitString(strArray,(char*)data, ", ");//以逗号切割 Split by comma
                for (int i = 0; i < 4; i++)
                {
                        strcpy(mystr_temp[i],strArray[i]);
                        Encoder_Now[i] = atoi(mystr_temp[i]);
                }

        }
        //10ms的实时编码器数据  10ms real-time encoder data
        else if ((strncmp("MTEP",(char*)g_recv_buff_deal,4)==0))
    {
        length = strlen((char*)g_recv_buff_deal)-5;
        for (uint8_t i = 0; i < length; i++)
        {
            data[i] = g_recv_buff_deal[i+5]; //去掉冒号 Remove the colon
        }
                data[length] = '\0';

                char* strArray[10];//指针数组 长度根据分割号定义  char 1字节   char* 4字节
Pointer array The length is defined by the split number char 1 byte char* 4 bytes
                char mystr_temp[4][10] = {'\0'};
                splitString(strArray,(char*)data, ", ");//以逗号切割 Split by comma
                for (int i = 0; i < 4; i++)
                {
                        strcpy(mystr_temp[i],strArray[i]);
                        Encoder_Offset[i] = atoi(mystr_temp[i]);
                }
        }
        //速度    Speed
        else if ((strncmp("MSPD",(char*)g_recv_buff_deal,4)==0))
    {
        length = strlen((char*)g_recv_buff_deal)-5;
        for (uint8_t i = 0; i < length; i++)
        {
            data[i] = g_recv_buff_deal[i+5]; //去掉冒号 Remove the colon
        }
                data[length] = '\0';

                char* strArray[10];//指针数组 长度根据分割号定义  char 1字节   char* 4字节
Pointer array The length is defined by the split number char 1 byte char* 4 bytes
                char mystr_temp[4][10] = {'\0'};
                splitString(strArray,(char*)data, ", ");//以逗号切割 Split by comma
                for (int i = 0; i < 4; i++)
                {
                        strcpy(mystr_temp[i],strArray[i]);
                        g_Speed[i] = atof(mystr_temp[i]);
                }
        }
}
```

- Deal_Control_Rxtemp：将接收到的数据进行筛选，符合通讯协议的就保存下来。
- Deal_data_real：将保存到的原始数据进行提取，重新组成一个新的打印格式。

1.3 实验现象

接线正确后，将程序烧录进主板中，复位后可以看见小车会前进2.5秒，后退2.5秒，右旋2.5秒，左旋2.5秒，然后停下2.5秒，又继续以上的状态切换动作。同时在串口助手里能看见打印的电机数值在不断的变化。

```
M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119

M1:1118,M2:1127,M3:1136,M4:1119
```